2026-10-15 · 第20节 · 学习任务管理系统 · 图文与代码练习

操作失败后，
后端怎样告诉网页发生了什么？

上一节，你写出了“修改优先级”“删除自己的任务”的 SQL，并接上了 Java 删除方法。这一节接着方法里的 `throw notFound()`，把失败的返回过程走完整。

> **今天解决的需求：**标题有误、登录失效、任务不存在、服务发生异常时，网页不能只显示“失败”，也不能显示成功。后端返回合适的状态码和消息，前端据此提示用户。

阅读方式：先看第 1 ~ 3 节的地图，再沿第 4 节删除失败的例子追踪代码。最后做第 6 节的书写练习。现在只整理课程，没有把这节记成已学完。

1. 先定位：今天补的是“结果怎样返回”

之前的主线是：网页 → Controller → Service → Mapper / MyBatis → MySQL。现在增加一条错误返回路径。统一异常处理器 `ApiExceptionHandler` 位于 Spring Boot 后端内部，负责把异常整理成 HTTP 响应。

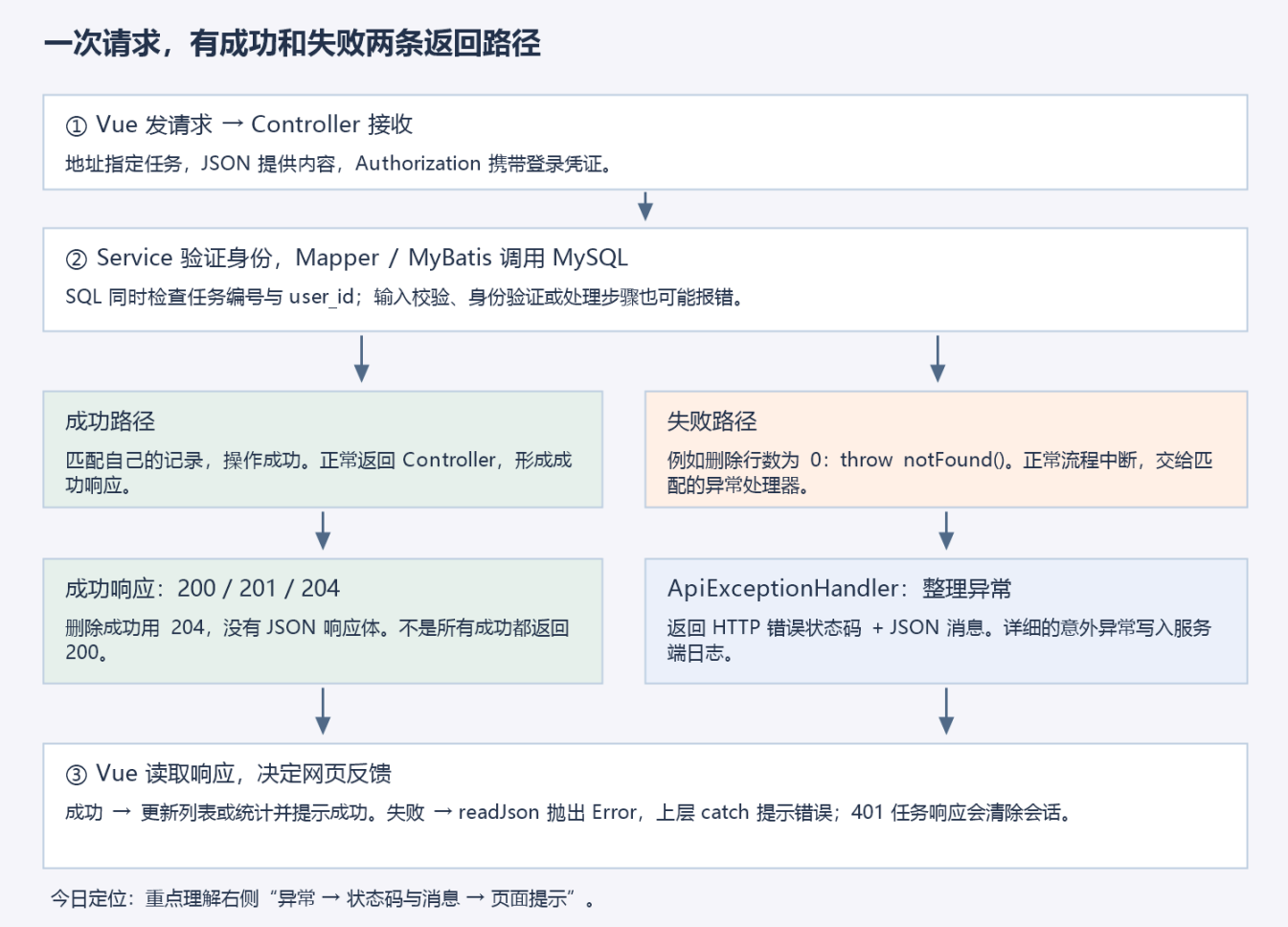


图 1：错误处理是整条功能链的返回分支。发生异常后，正常路径中后续尚未执行的语句不会继续运行；Spring 寻找匹配的异常处理器。

为什么要单独写异常处理器?

新增、修改、删除都有可能失败。如果每个 Controller 都重复写相同错误响应，维护容易不一致。统一处理器让 Service 表达“发生了什么”，再由集中代码决定“怎么返回给网页”。

这不是新建一套业务，也不是把正常请求全部绕到异常处理器。只有匹配的异常发生时，它才负责生成错误响应。

> **本节的前后关系：**前一节决定“允许改哪条任务”；本节决定“不允许或失败时怎样返回”；后续前端课程决定“接收响应后怎样恢复页面、提示用户”。

2. 状态码与消息：一份响应里的两部分

后端返回的 HTTP 响应包含状态码和响应体。状态码用于分类结果，响应体可以提供更具体的文字。例如删除不属于自己的任务：

```
HTTP 状态码：404
响应体（JSON）：
{"message":"任务不存在或不属于当前用户"}
```

这里不是“JSON 中有个 404 字段”。404 是 HTTP 响应状态，前端通过 `response.status` 读取；`message` 是 JSON 字段，前端解析后通过 `data.message` 读取。

同一次任务操作，不同的失败原因

400 · 请求内容不符合要求

例：空标题、非法状态或无法解析的 JSON。处理：修正输入；后端也必须校验。

401 · 登录身份未通过验证

例：密码错误，缺少、无效或过期 token。处理：检查账号密码或重新登录。

404 · 任务不存在或不可操作

例：任务已删除，或它属于别人。本项目统一返回 404，不透露他人任务是否存在。

500 · 服务端发生意外错误

例：处理时数据库连接失败引发异常。用户看通用提示，开发者看后端日志。

两个信息要分开：response.status 是 HTTP 状态；data.message 是 JSON 里的说明。

服务完全关闭时可能没有 HTTP 响应；网络错误不能一律当成后端 500。

状态	本项目里的触发例子	用户怎么处理
400	请求标题空白、状态值非法、JSON 格式错误	检查并修正输入
401	密码错误；缺少、无效或过期 token	检查账号密码或重新登录
404	任务已被删除，或当前用户操作别人的任务	刷新列表；不能操作他人任务
500	后端出现未预料到的处理错误，例如数据库连接失败引发异常	稍后重试；开发者查看日志排查

补充认识：200 常表示请求成功；201 表示创建成功；204 表示成功但没有响应体。项目注册重复邮箱会返回 409。今天先重点理解四类错误，不需要背所有状态码。

权限不足也可以采用 403；但本项目对“任务不存在”和“属于别人”统一返回 404，减少泄露任务是否存在。这个规则特指当前任务接口。

> **服务断开不等于一定收到 500。**如果 Spring Boot 已关闭，浏览器可能根本收不到 HTTP 响应；经过 Vite 代理时也可能收到代理生成的错误。不要只凭一个数字就断定根因。

3. 这一节让网页哪里更可靠?

页面不一定多一个菜单。变化在于同一个按钮遇到失败时的反馈：不错误地显示成功，让用户知道接下来怎么办。

网页操作	正确行为	当前实现的位置
任务名称留空，点创建	网页先提示“请输入任务名称”，不发创建请求	App.vue 的 createTask，211 行附近
绕过网页提交空标题	后端仍检查，返回 400；不会新增空任务	TaskRequest + @Valid + 异常处理器
登录凭证已失效，操作任务	任务请求收到 401，清除会话并提示重新登录	App.vue 的 apiRequest，75 行附近
删除不存在或他人的任务	返回 404；前端显示错误，数据库不会删除他人记录	TaskService.delete → notFound → 统一处理器
服务处理出现意外错误	返回通用服务错误；详细异常保留在后端日志	异常处理器 unexpected；前端 readJson

当前项目已经有这些处理代码，本课要读懂并做一个小改动。此前验收空标题网页提示，只证明前端拦截成功；后端校验由直接接口测试验证，不能把两者混为一谈。

当前前端对 400、401、500 使用自己的提示文案，404 则通常采用后端 `message`。因此只修改后端 400 文案，网页提示不一定跟着变；必须追踪前端如何处理这个状态。

4. 从你写过的删除方法，追踪到网页提示

4.1 SQL：没有找到可删除记录

```
DELETE FROM tasks
WHERE id = #{id} AND user_id = #{userId};
```

假设任务 12 属于用户 3，而当前登录用户是 8。任务编号匹配，但用户编号不匹配，因此删除行数是 0。记录保持不变。

4.2 Service：停止正常流程并抛出异常

```
public void delete(Long id, String authorization) {
    Long userId = currentUserId(authorization);
    if (mapper.delete(id, userId) == 0) {
        throw notFound();
    }
}
```

你已经写过这段。现在展开 `notFound()`：

```
private ResponseStatusException notFound() {
    return new ResponseStatusException(
        HttpStatus.NOT_FOUND,
        "任务不存在或不属于当前用户"
    );
}
```

`new ResponseStatusException(...)` 创建异常对象，携带状态和原因；`return` 把这个异常对象交回调用处；前面的 `throw` 才真正抛出它并中断当前处理。只创建异常对象而不 throw，不会自动中断。

`HttpStatus.NOT_FOUND` 是 404 的 Java 名称。不要先背完整类名，先看出这是“状态 + 说明文字”的组合。

4.3 统一处理器：把异常变成响应

```
@ExceptionHandler(ResponseStatusException.class)
public ResponseEntity<Map<String, String>> responseStatus(
        ResponseStatusException error) {
    String message = error.getReason() == null
            ? "请求失败" : error.getReason();
    return ResponseEntity.status(error.getStatusCode())
            .body(Map.of("message", message));
}
```

写法	在这次请求里的含义
@RestControllerAdvice	标在处理器类上，让 Spring 使用它集中处理 Controller 请求中的异常
@ExceptionHandler(...)	这个方法负责匹配指定类型的异常
error.getStatusCode()	从异常取得 404
error.getReason()	取得“任务不存在或不属于当前用户”
Map.of("message", message)	准备 message 字段，Spring 将其转换成 JSON
ResponseEntity	让我们指定 HTTP 响应的状态和响应体

这段代码可同时处理携带 401、404、409 等状态的 ResponseStatusException，不是每个状态都要写一个重复方法。校验异常和意外异常则由其他匹配的方法处理。

4.4 前端：响应失败时进入 catch

```
// App.vue: readJson 的处理摘录
if (!response.ok) {
    // 400、401、500 等有专门分支
    throw new Error(data?.message || "请求失败，请稍后再试");
}

// removeTask 的处理摘录
try {
    await apiRequest(`/api/tasks/${task.id}`, { method: "DELETE" });
    await loadTaskData();
    ElMessage.success("任务已删除");
} catch (error) {
    ElMessage.error(error.message);
}
```

这是分段摘录，不是完整函数。404 会使 apiRequest 内部的 readJson 抛出 Error，因此 await 后面的成功提示不会执行，转入 catch 显示错误。

原生 fetch 收到 404、500 时通常仍会得到一个 Response，不会仅因为状态是错误就自动进入 catch。项目的 readJson 主动检查 `response.ok` 并抛出错误，才把失败交给上层 catch。

5. 400、401、500 分别在哪一层产生?

400：请求内容没有通过规则

```
// TaskRequest.java
@NotBlank @Size(max = 200) String title

// Controller 方法参数
@Valid @RequestBody TaskRequest request
```

RequestBody 转换请求数据；Valid 检查规则。空白标题会触发校验异常，由处理器的 validation 方法返回 400 和消息。JSON 根本无法解析时，由 malformedInput 分支处理。

401：当前身份未通过验证

```
throw new ResponseStatusException(
    HttpStatus.UNAUTHORIZED,
    "Token 无效或已过期"
);
```

项目中 AuthService 负责验证凭证；任务 Service 通过它确认用户。无效凭证不能读写任务。多个条件同时不合法时，不要依赖它们固定的报错顺序；练习每次只制造一种错误。

500：未预料到的服务器错误

```
@ExceptionHandler(Exception.class)
public ResponseEntity<Map<String, String>> unexpected(
        Exception error) {
    log.error("API request failed", error);
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(Map.of("message", "服务暂时不可用，请稍后重试"));
}
```

这里保留详细日志，给页面通用提示；不是忽略异常，也不能把所有错误都当作 500。已有更具体的异常处理器优先匹配对应错误。

> 不要为练习 500 修改数据库密码、破坏表或在正式项目加一个故意崩溃的公开接口。今天通过代码与参考响应理解，隔离测试在后续测试课补充。

对应的真实文件与行号

行号依据 2026-10-15 本机代码；后续修改可能变化。前端根目录是桌面 09-ElementPlus-后台Demo，后端根目录是桌面 14-SpringBoot-JWT-Demo。

文件	当前行号	阅读重点
后端 src/main/java/com/learning/jwtapi/TaskService.java	49、73	删除结果判断、404 异常对象
同目录 ApiExceptionHandler.java	16、19、25、34、39	集中处理器，以及业务、校验、格式、意外异常的分支
同目录 TaskRequest.java	7 ~ 11	输入规则
同目录 AuthService.java	24 ~ 43	无效 token、缺少凭证的 401
前端 src/App.vue	39、61、255	readJson、apiRequest、removeTask 的失败处理

6. 今天的代码练习：先模仿，再做一个小变化

练习写在草稿里，不直接粘进正式类。下面不要求你默写整个处理器，也不把“看过代码”记成“已经会写”。

跟写 A：创建并抛出一条 404 异常

```
if (mapper.delete(id, userId) == 0) {
    throw new ResponseStatusException(
        HttpStatus.NOT_FOUND,
        "任务不存在或不属于当前用户"
    );
}
```

它与原先的 `throw notFound()` 表达同一意图，只是把辅助方法的内容展开。不要在正式项目同时保留两个重复检查。

跟写 B：写一个最小的响应构建方法

```
// 草稿辅助方法；不是项目中已有的方法名
public ResponseEntity<Map<String, String>> buildNotFoundResponse() {
    return ResponseEntity.status(HttpStatus.NOT_FOUND)
            .body(Map.of("message", "任务不存在或不属于当前用户"));
}
```

这个方法只演示“构建响应”，单独写出来不会自动处理异常；实际项目还需要类上的 Advice 和方法上的 ExceptionHandler 等连接关系。先看清 return 返回的是完整响应，包含 404 和 message。

相近练习：把上面改成“输入有误”的响应

在草稿里另写一个 `buildBadRequestResponse` 方法：状态改成 `HttpStatus.BAD_REQUEST`，文字改成“请检查任务名称”。参考上面方法的结构，尝试自己输入，再看参考答案。

参考答案：400 响应构建

实际项目的小改动：让 404 提示更容易理解

跟写通过后，再在我的检查下修改 TaskService 的 notFound 原因文字为“任务不存在或无法访问，请刷新任务列表”，保留 NOT_FOUND 状态和 SQL 归属条件。修改前先找到唯一位置；不要替换所有出现“请求失败”的字符串。

这项正式改动目前只是课堂任务，尚未执行。修改后需要重新启动使用新代码的后端再验证，浏览器刷新不会自动更新已经启动的旧 JAR。仅更改草稿不会影响页面。

今天亲手完成的变化虽然小，但它贯穿 Service → 异常处理器 → HTTP → 前端消息，而不是机械地修改一个孤立字符串。

7. 验证、时间与完成标准

先做两个不改数据的检查

网页留空标题点创建，再看浏览器开发者工具 Network：应有前端提示，没有新的 POST 请求。它验证前端拦截，不是后端 400。

后端启动后，在一个空闲 PowerShell 执行下面命令。它不带 token 查询任务，只读取响应，不新增或删除数据：

```
try {
    Invoke-RestMethod -Uri "http://localhost:8084/api/tasks" -Method GET
} catch {
    if ($_.Exception.Response) {
        [int]$_.Exception.Response.StatusCode
    } else {
        $_.Exception.Message
    }
}
```

服务正常运行且没有凭证时，预期输出 401。连接失败时显示网络错误，不能直接认定后端返回了 500。这段用于 Windows PowerShell；在未启动后端时先启动，不重复抢占 8084。

跟写与改动后再做一轮受控验证

沿用已有 api-smoke-test.ps1 检查后端 400、401、404 和任务 CRUD；运行前由我确认后端启动的是新代码。脚本会创建随机测试账号、创建并清理自己的测试任务，保留测试账号，不动原有任务。今天准备讲义时没有重新运行，不把之前的测试结果当作本节新验收。

404 文案改动后，需要直接请求查看 JSON 并确认前端处理该 message；不能只凭编译通过就记录页面提示通过。500 本课先做代码追踪，不要求破坏环境制造故障。

环节	时间	今天做什么
完整地图	20 分钟	先读第 1 ~ 3 节，知道错误返回链在哪里
概念与设计	25 分钟	区别状态码、JSON 消息、网络错误
代码追踪	25 分钟	沿删除失败追到前端 catch
跟写	55 分钟	异常与响应片段，检查后做小改动
相近练习	25 分钟	从 404 响应改写为 400 响应
验证	15 分钟	只检查本节相关行为，不重复完整点击流程
复盘	15 分钟	记录哪些能解释、能跟写、能自己改

本节完成标准

- 能沿“删除行数为 0 → throw → 统一处理 → 404 + JSON → catch 提示”说明发生什么。
- 在参考帮助下写出异常或响应片段，能把 404 例子改成 400。
- 完成一次有位置、有目的的小改动和对应验证；如果尚未完成就如实记录，下一次从这里续上。

> 进度记录：上一节图解阅读和经纠错的 SQL、删除方法草稿已完成；独立实现整个功能尚未证明。本节资料已准备，学习和验证待开始。日期是安排，不能用“网页早已能用”代替当天学习。

建议以后减少每天新增术语，固定用一个任务例子贯穿全链。框架图每天保留一张，并突出当天变化，不每天重讲整本资料。统一交付、构建与演示仍按月底目标保留。